

Agent Validation Visualization and Interaction

Trevor Hardy

The set of visualizations and interactions is designed to provide those that are designing agents to understand both the workflow of agents in general as well as to understand how the particular agent that has been implemented (and is being called to realize these visualizations) is functioning and to help validate whether it is functioning as expected.

The structure of this tooling is based on the workflow of the agent itself and the various functions that constitute the software architecture of the agent. In this regard, it is something of both a tutorial for how the agent is expected to function as well as interactive testing for the specific functions of the agent. The draft GUI shown in this document shows the visualizations as tabs in a window but, due to the interactivity between various functions, it is likely that it will be necessary to implement this more like a linear set of settings and interactions, *a la* a Windows wizard. For example, choosing a market clearing price happens in “F7 - Price Response” but also impacts “F8 - Control Signal Translation”. Users will need to explicitly choose to go back to previous steps to adjust the necessary input with the understanding that all subsequent steps will be altered in some way.

Relatedly, this proposed, step-oriented interaction is likely to not be preferred by those that are trying to evaluate the performance of a particular agent they have implemented. Due to the multi-step analysis process and the impacts of choices made in earlier steps having significant impact on later steps, it is almost certain that users will want to, say, adjust the market clearing price in F7 and see, as they adjust the price, what impact this has on the control signals. Right now these are in two separate views, making an interactive evaluation of the impact of price on control signals impossible. Before implementing any of these visualizations and interactions more thought needs to be given to what we expect users to want to do and how to best facilitate those interactions. Maybe all the inputs can be defined in a separate pane from the large number of visualizations? Maybe important inputs from previous steps can be brought forward to the appropriate view as well? Maybe multiple views can be combined?

To help users evaluate the behavior of the agent and validate its correctness, it will be important to allow users to select a portion of the population of devices to visualize. At times, this maybe just a a single device to allow for concentrated evaluation. At other times, looking across the population as a whole will be necessary. There are a number of GUI elements used across the visualization steps that achieve this. I don’t think I would go so far as to say these are ideal; further thought should be given as to best achieve the selection and grouping function that are commonly needed when evaluating large populations.

Very relatedly, there are a lot of signs that point to advantages to using a database to achieve the necessary interactions. Though we’ve traditionally stored customer definitions in a JSON file there’s no reason it couldn’t be a database. Similarly, storing the results of the agent calculations in a database likely makes a lot of sense. Off the top of my head, I suspect all of the data we need could be tabular, making something like Postgres an easy option. Furthermore, once the data is stored in a database, there are a large number of existing tools out there that are designed to pull data from a database and visualize it. Assuming the right tooling exists, it seems like we could implement this as a web service and run it from an internal website pretty easily. We would have to figure out how to allow users to choose the specific versions of functions they want to use at each step. But it might work very well for us. In particular, this would likely be an easier way for the economists to play around with the agents then having them learn Git, Python, and potentially Docker.

Lastly, these visualizations and interactions are effectively largely qualitative in nature. This is the level of inspection and validation that DSO+T generally undertook, just in a fancier and prettier package. It is not expected this tooling will help identify edge cases, outliers, or less than obvious bugs in the existing agent code. In this regard, it does not achieve my stated goal of trying to prove or provide a high degree of confidence that the code is working as expected. This validation approach is much more “well, it looks like it’s working as expected.” Right now I want us to do better but this seems like a good start that will allow us to better understand how given agent code is working and provide a gross evaluation of the LLM-generated code.

(Jessica writes about the customer generation process.)

(Need to include something about which devices will be considered controllable and thus will have agents associated with them.)

The primary output of the customer generation process is a customer dictionary that defines various GridLAB-D and other customer parameter values. This customer dictionary is directly tied to the underlying GridLAB-D model that will be used in the simulation.

The inspection and visualization tool below provides a means of understanding the population that has been generated. It provides a means of selecting and grouping customers to inspect various parameter distributions.

Customer Generation	Model State	Flexibility Estimation	Preference Curve	Bid Formulation	Price Response	Control Translation
---------------------	-------------	------------------------	------------------	-----------------	----------------	---------------------

Load Dictionary

There’s got to be an existing database query and visualization tool we can. use instead of writing our own.

Under normal conditions, we would just run the GridLAB-D to provide a means of defining the state of the devices being modeled in GridLAB-D. This is a comprehensive means of providing the necessary data needed by the agents but it is also largely outside our control and thus makes validation much more difficult.

Instead, the tooling provides a means of setting specific model parameters in such a way so as to create the conditions needed to evaluate the performance of the agent. These parameters will be device specific.

The output of the this model state definition is a dictionary that will be written to a file or to memory. The model state is used in the following process of the agent operation where the flexibility of each device is estimated.

Customer Generation

Model State

Flexibility Estimation

Preference Curve

Bid Formulation

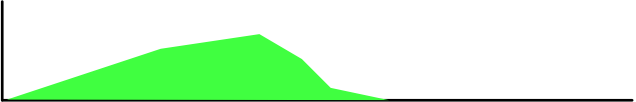
Price Response

Control Translation

Number of customers: 120

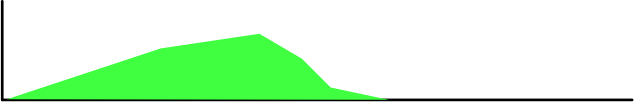
HVAC

Number of HVACs: 100
Number of HVAC agents: 80

	mean	st. dev.	
indoor temperature	150	150	
outdoor temperature	150	150	

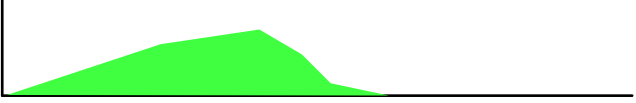
Water Heater

Number of water heaters: 100
Number of water heater agents: 60

Upper tank temperature	150	150	
Lower tank temperature	150	150	

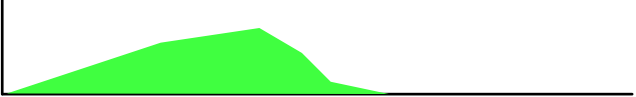
EV Charger

Number of EV chargers: 100
Number of EV charger agents: 60

EV fraction available to charge	150	150	
EV starting SOC	150	150	

Battery

Number of batteries: 100
Number of battery agents: 60

Normalized available SOC	150	150	
--------------------------	----------------	----------------	---

Create Model State Dictionary

Visualize Model States

An estimate of the flexibility of each device is necessary to allow the agent to understand the resources it has available to bid into the market. This flexibility estimate is based on the device state and its physical or operational limits. The device state is defined the previous step and the flexibility estimate for each device is calculated here.

The GUI provides a means of selecting the population of devices to evaluate, allowing users to select specific devices to inspect the flexibility estimation process.

Customer Generation

Model State

Flexibility Estimation

Preference Curve

Bid Formulation

Price Response

Control Translation

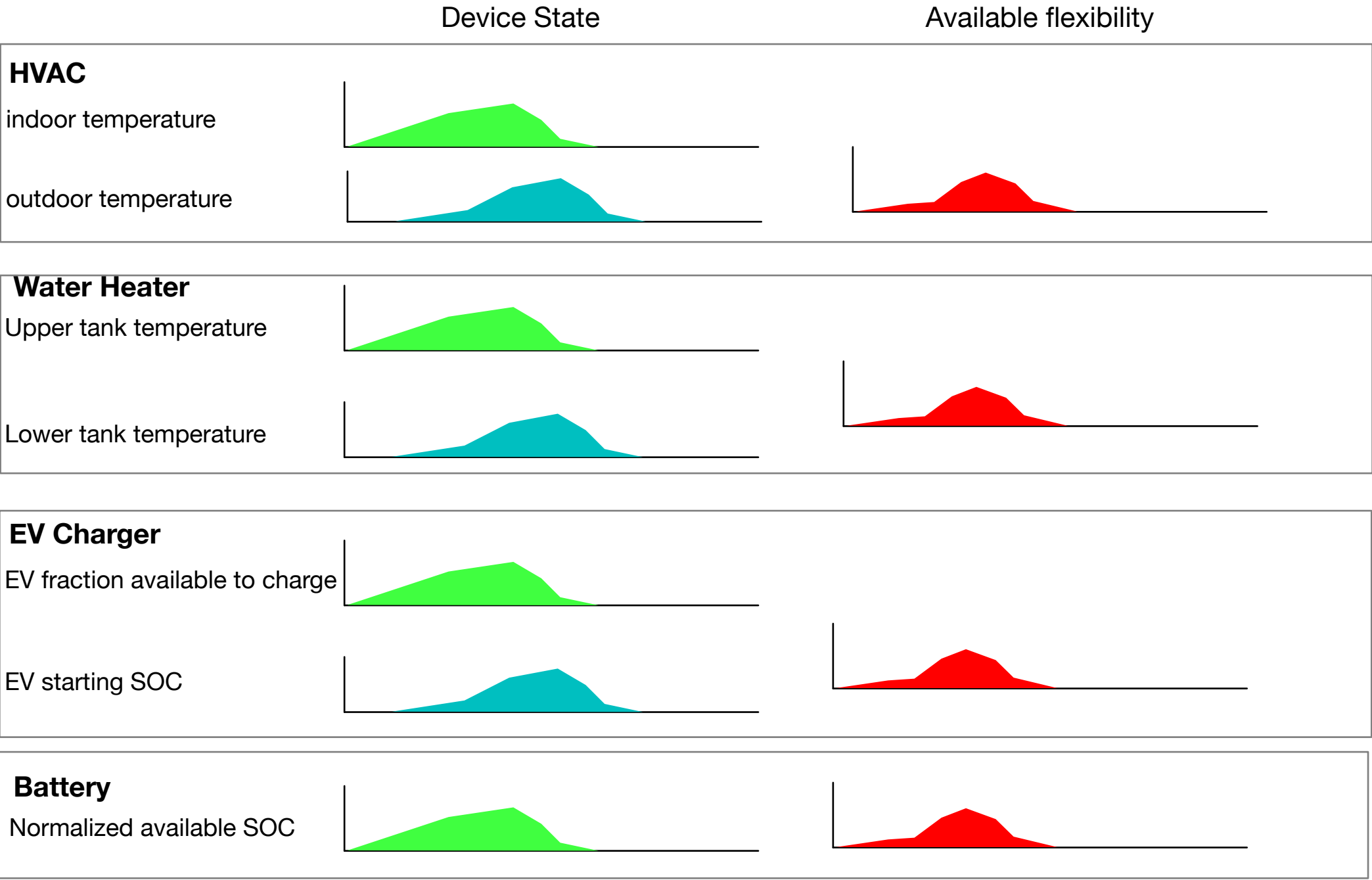
Number of customers: 120

Customer Selection

- ☒ customer_1
- ☒ customer_2
- ☒ customer_3
- ☒ customer_4

- ☒ HVAC
- ☒ Water Heater
- ☒ EV Charger
- ☒ Battery

Estimate Device Flexibility



Customer preference is independent of device state; it is expressed through a per-device slider setting where “0” represents prioritization of amenity and “1” represents prioritization of cost minimization.

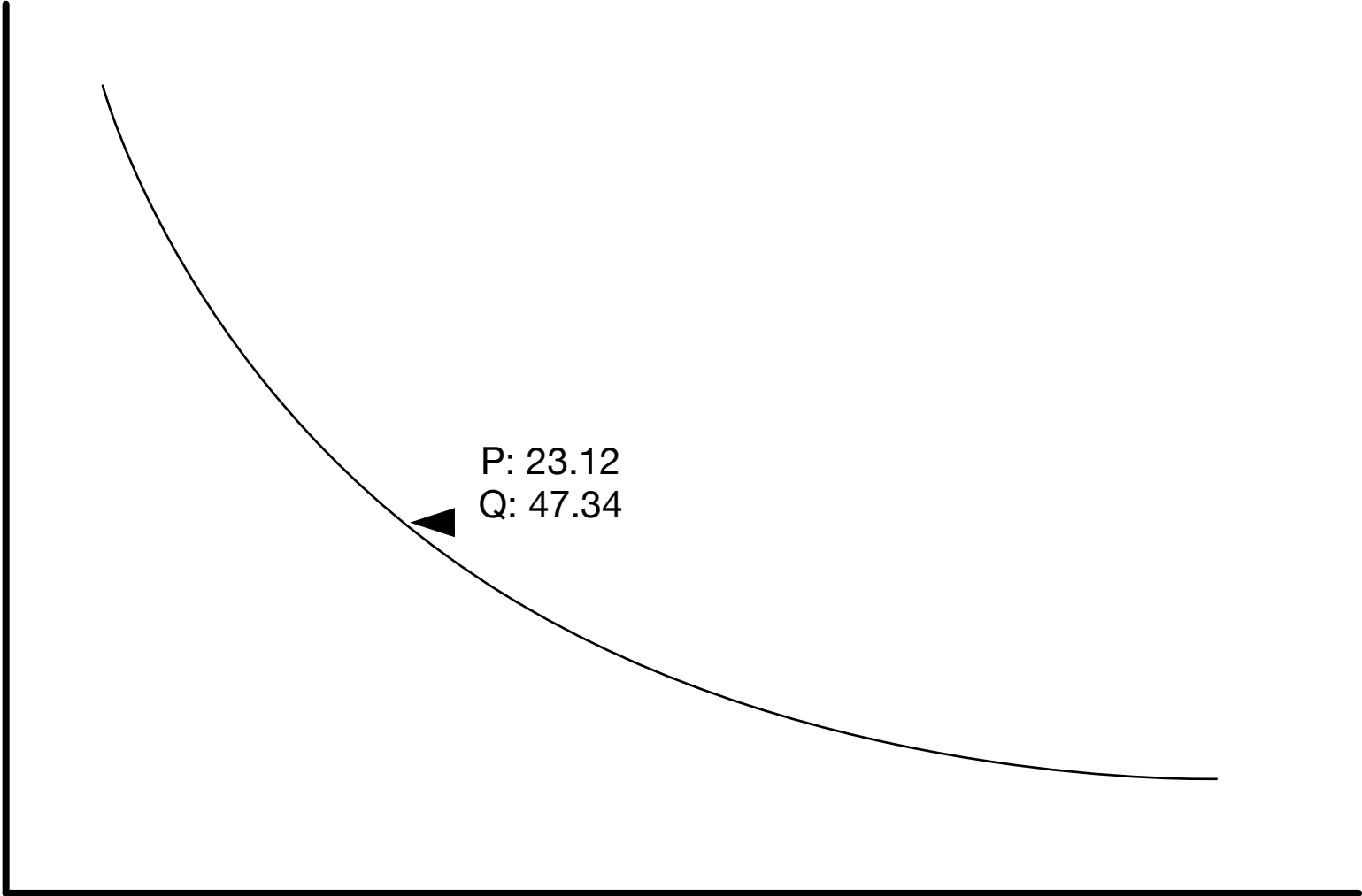
This preference is expressed through setting values for isoelastic curve parameters through a preference correlation function. This function is implemented in agent code and is used to define the relationship between various customer parameters (e.g. house size, income level, location of residence) to their device slider setting. These functions are custom made and have an input of the customer dictionary (containing all the customer parameters) and an output of the same customer dictionary with the addition of the slider setting, a scale, and elasticity parameter values defined. The GUI below provides a way of selecting from the existing correlation functions defined in the appropriate Python file.

In addition to selecting the correlation function, users can determine the population of devices and population customers to include in the aggregate preference curve being plotted. Mousing over the curve would show the indicated price and quantity points

- Customer Generation
- Model State
- Flexibility Estimation
- Preference Curve
- Bid Formulation
- Price Response
- Control Translation

Customer Preference

Price (\$/kWh)



Quantity (kWh)

Preference Correlation Function

- income_based
- income_and_neighbor
- dan_magic_1
- sadie_knows_best

Device

- ☒ HVAC
- ☒ Water Heater
- ☒ EV Charger
- ☒ Battery

Customer Selection

- ☒ customer_1
- ☒ customer_2
- ☒ customer_3
- ☒ customer_4

Num customers graphed: 120

Average Scale: 120

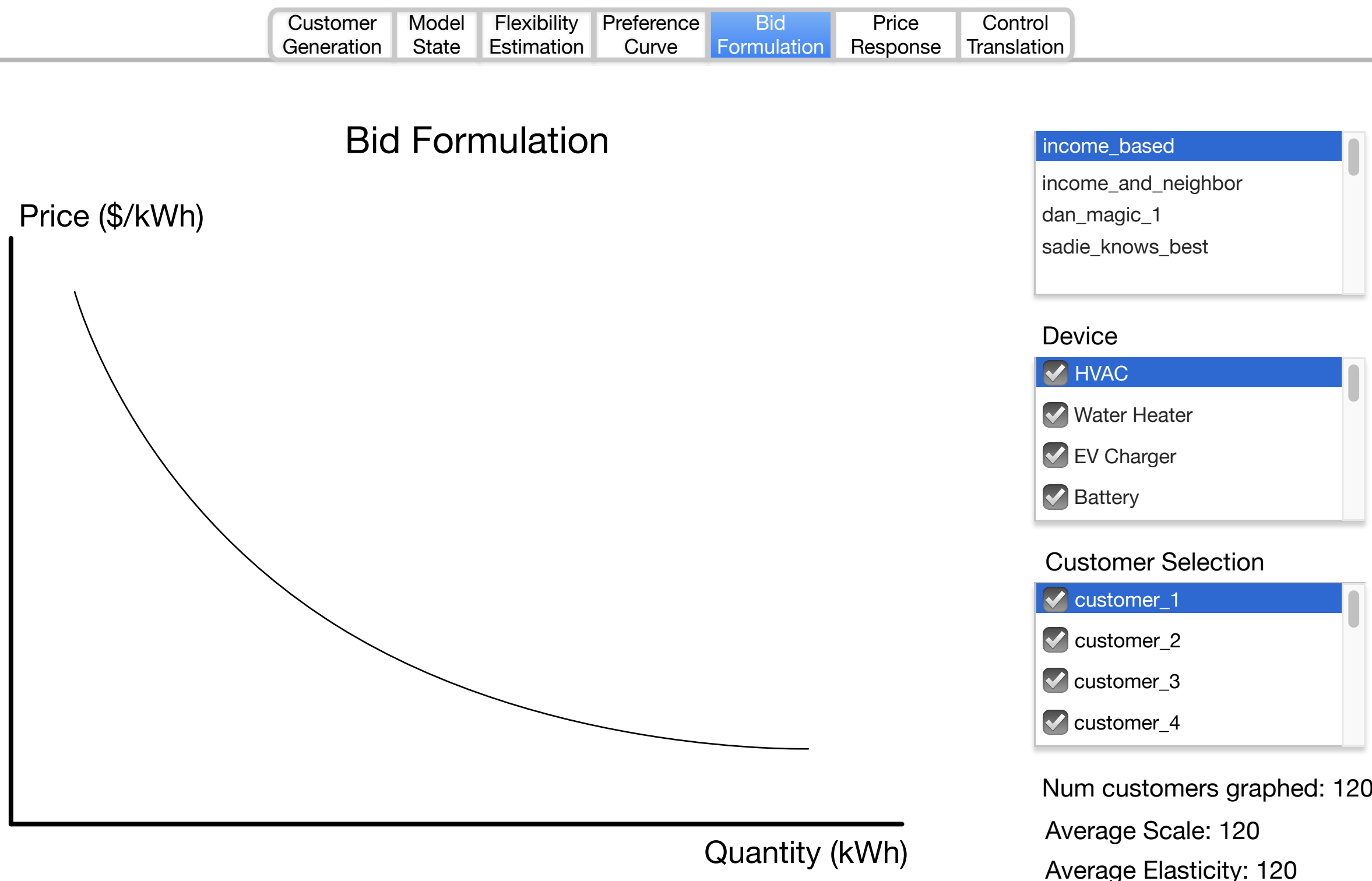
Average Elasticity: 120

Bid formulation is the process by which a customer's preferences (defined in the preference curve) are translated into bids sent to the market based on the estimated flexibility of the device for which the agent is submitting a bid. It is the preference read through the device's flexibility estimate into a form the market requires.

Many of the parameters that might be used in the bid formulation are static and have been defined when the customer dictionary was generated. This includes things like a nominal heating and cooling set point (for HVAC systems), water heater set point, and battery emergency reserve SOC.

The visualization of the bid formulation will be market dependent. In particular, markets that resolve over multiple periods (such as day-ahead energy) will need more complex visualizations and user inputs. The image below is for the real-time energy market.

It may make more sense to integrate this with one or more of the previous views; it would be messy.

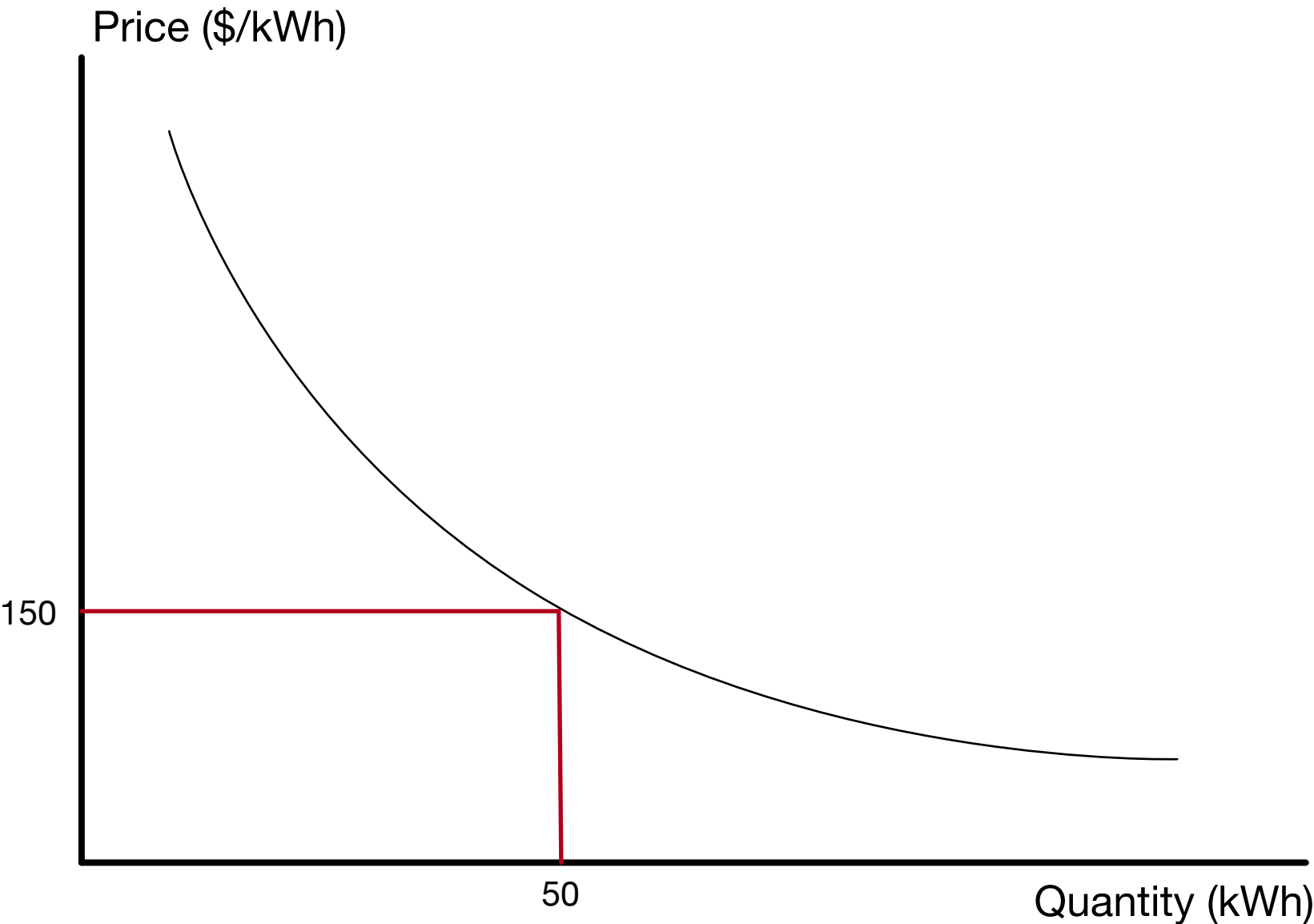


The price-response functionality is a simple market operation. A price from a higher-level market (wholesale, retail) is fed into the preference curve or the bid curve and from that, an energy consumption for that period is found.

Like other similar functionality, there is functionality to select the devices shown in the aggregated curve. There is also functionality to determine if the preference curve (price-responsive devices only) or bid curve (retail market participant). Users can adjust the market clearing price to see what the selected population's cleared energy would be.

- Customer Generation
- Model State
- Flexibility Estimation
- Preference Curve
- Bid Formulation
- Price Response
- Control Translation

Price Response



- ☒ Preference Curve
- ☐ Bid Curve

- Device
- ☒ HVAC
- ☒ Water Heater
- ☒ EV Charger
- ☒ Battery

- Customer Selection
- ☒ customer_1
- ☒ customer_2
- ☒ customer_3
- ☒ customer_4

Num customers graphed: 120

150

Market Price (\$/kWh)

50

Cleared Energy (kW)

Based on the results from the price response calculation, each device will have a target operation expressed in terms of the market product (e.g. energy). To adhere the the expressed preference or bid, this market product must be translated into specific control actions for the device(s) under control. For HVAC devices participating in a an energy market, the control action is a set point; for batteries, it is a charging or discharging power. Note that devices generally operate at a fixed power and the operation time is modulated; batteries are a significant exception.

Devices that store thermal energy are controlled through set point change and to estimate the energy consumption for the market period via some form of device model. This model must be able to translate the target energy consumption into a device set point by simulating the behavior of the device for the duration of the market period.

Customer
Generation

Model
State

Flexibility
Estimation

Preference
Curve

Bid
Formulation

Price
Response

Control
Translation

Control Signal Translation

Number of customers: 120

Customer Selection

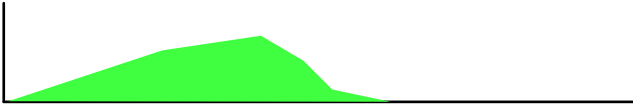
- ☒ customer_1
- ☒ customer_2
- ☒ customer_3
- ☒ customer_4

- ☒ HVAC
- ☒ Water Heater
- ☒ EV Charger
- ☒ Battery

Calculate Control Signals

HVAC

Setpoint



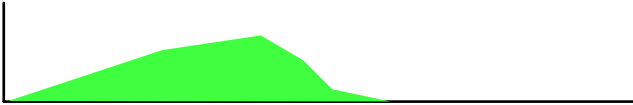
Fraction of devices at maximum 0.2

Fraction of devices at minimum 0.1

Effective SOC change +0.01

Water Heater

Setpoint



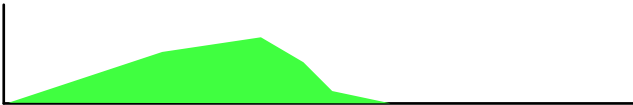
Fraction of devices at maximum 0.2

Fraction of devices at minimum 0.1

Effective SOC change +0.01

EV Charger

Charging Power



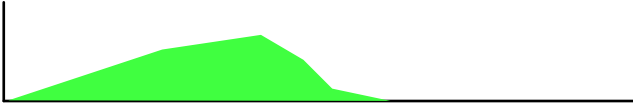
Fraction of devices at maximum 0.2

Fraction of devices at minimum 0.1

Effective SOC change +0.01

Battery

Power



Fraction of devices at maximum 0.2

Fraction of devices at minimum 0.1

Effective SOC change +0.01